



taller-cicd-a00401428

📖 Materia	📖 <u>INGENIERIA DE SOFTWARE V</u>
⚙️ Estado	Pendiente
📅 Fecha	02/09/2026

AI used: **Gemini**

<https://share.gemini.google/A5gmgoNLQz0i>

The project runner is being hosted on ubuntu vm using the following configuration:

repo github

<https://github.com/ingesoft5/taller-cicd-a00401428.git>

Integración y pasos a realizar

- **Aislamiento en los Dockerfiles:** Al construir los multi-stage builds primero, aíslas los problemas de compilación. Puedes asegurar que Maven empaqueta correctamente el **.jar** de Spring Boot y que Node/Vite generan el **dist** antes de que Nginx entre en juego.
- **Integración con Docker Compose:** Una vez que las imágenes individuales existen, el **docker-compose.yml** te permite resolver el enrutamiento y las redes internamente. Es el momento ideal para verificar que la red compartida funciona y que la SPA puede hablar con la API REST de Spring Boot sin problemas de CORS.

- **Traducción directa al Pipeline:** Cuando llegues a armar el `.github/workflows/ci-cd.yml`, los jobs de construcción serán prácticamente una transcripción de los comandos que ya validaste en tu terminal local.

Aislamiento en los Dockerfile

Backend

Un enfoque **multi-stage** dividido en dos partes. La primera etapa compila el código, descarga las dependencias y ejecuta las pruebas unitarias. La segunda etapa toma únicamente el archivo `.jar` compilado y lo coloca en una imagen ligera de producción, dejando atrás todo el código fuente y las herramientas de construcción (como Maven) para reducir drásticamente el tamaño final de la imagen.

```
# =====
# Etapa 1: Construcción y Pruebas (Builder)
# =====
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

# 1. Copiar primero el descriptor del proyecto
COPY pom.xml .

# 2. Descargar las dependencias fuera de línea
# Esto aprovecha el caché de capas de Docker. Si el pom.xml no cambia,
# Docker no volverá a descargar todo internet en las siguientes construcciones.
RUN mvn dependency:go-offline -B

# 3. Copiar el código fuente de la aplicación (y las pruebas)
COPY src ./src

# 4. Compilar el proyecto, ejecutar pruebas unitarias y empaq
```

```

uetar
# Al usar H2 en memoria, las pruebas del repositorio y servicios se ejecutarán
# sin necesidad de levantar bases de datos externas en este paso.
RUN mvn clean package -B

# =====
# Etapa 2: Entorno de Ejecución (Runtime)
# =====
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Por seguridad, creamos un usuario sin privilegios de root para ejecutar la app
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

# Copiar el archivo .jar generado en la etapa 'builder'
# Se usa un comodín (*.jar) para no depender del nombre o versión exacta en el pom.xml
COPY --from=builder /app/target/*.jar app.jar

# Exponer el puerto en el que escucha Spring Boot por defecto
EXPOSE 8080

# Punto de entrada para iniciar la aplicación
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

```

Frontend

Multi-stage para el frontend y la configuración correspondiente de `nginx.conf`, indispensables para que una SPA en React no devuelva un error 404 al recargar páginas con rutas internas.

```

# =====
# Etapa 1: Construcción (Builder)
# =====
FROM node:20-alpine AS builder

WORKDIR /app

# 1. Copiar manifiestos de dependencias primero para optimiza
r la caché
COPY package*.json ./

# 2. Instalar dependencias usando clean install (reproducible
y rápido)
RUN npm ci

# 3. Copiar el resto del código fuente del proyecto
COPY . .

# 4. Compilar la aplicación TypeScript a estáticos optimizado
s
# En Vite, esto genera por defecto la carpeta /app/dist
RUN npm run build

# =====
# Etapa 2: Servidor Web de Producción (Runtime)
# =====
FROM nginx:alpine

# 1. Eliminar la configuración predeterminada de Nginx
RUN rm -rf /etc/nginx/conf.d/default.conf

# 2. Copiar nuestra configuración personalizada para la SPA
COPY nginx.conf /etc/nginx/conf.d/default.conf

# 3. Copiar los archivos estáticos generados por Vite en la e

```

```
tapa builder
COPY --from=builder /app/dist /usr/share/nginx/html

# Exponer el puerto estándar HTTP
EXPOSE 80

# Iniciar Nginx en primer plano para mantener el contenedor activo
CMD ["nginx", "-g", "daemon off;"]
```

Nginx

```
server {
    listen 80;
    server_name localhost;

    # Directorio donde se encuentran los archivos compilados de Vite
    root /usr/share/nginx/html;
    index index.html;

    # 1. Enrutamiento de la SPA: si una ruta no existe como archivo físico,
    # redirige a index.html para que React Router tome el control
    location / {
        try_files $uri $uri/ /index.html;
    }

    # 2. Proxy inverso hacia el Backend en Spring Boot
    # El nombre de host 'backend' coincide con el nombre del servicio en docker-compose
    location /api/ {
        proxy_pass http://backend:8080/;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
```

```

        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forward
ed_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Manejo de páginas de error
    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}

```

Docker Compose

Une ambas piezas y garantiza que el frontend (y Nginx) no empiece a recibir tráfico o intentar rutear hacia el backend hasta que Spring Boot haya levantado completamente y responda en su endpoint de productos.

```

version: '3.8'

services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: springboot-backend
    ports:
      - "8080:8080"
    networks:
      - app-network
    # Healthcheck verificando el endpoint específico solicita

```

```

do
  healthcheck:
    # Alpine Linux usa wget por defecto (no curl)
    test: ["CMD-SHELL", "wget --spider -q http://localhost:
8080/api/products || exit 1"]
    interval: 15s      # Cada cuánto se hace la prueba
    timeout: 5s        # Cuánto espera a que responda la pe
tición
    retries: 5         # Cuántas veces falla antes de marca
rlo "unhealthy"
    start_period: 20s  # Tiempo de gracia para que la JVM d
e Spring Boot inicie

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: react-frontend
    ports:
      - "80:80"
    networks:
      - app-network
    depends_on:
      backend:
        condition: service_healthy # Espera explícitamente a
que el healthcheck pase

  networks:
    app-network:
      driver: bridge

```

Runner set-up

```

jobs :
  test - backend :

```

```
runs - on : ubuntu - latest
steps :
- uses : actions / checkout@v4
- name : Run Maven Tests
run : cd backend && mvn clean test
```

Cuestionario de Analisis & Conceptos DevOps

Inmutabilidad y Multi-Stage:

1. **Inmutabilidad y Multi-Stage:** ¿Por qué no se debe incluir el SDK completo de Java (JDK) ni el compilador Maven en la imagen Docker final de producción? ¿Qué impacto tiene esta decisión en el tamaño de la imagen y en la superficie de ataque (CVEs)?

Excluir el JDK y Maven de la imagen final es vital por **seguridad, velocidad y costos**. Al dejar únicamente el JRE, reduces drásticamente el peso del contenedor (de casi 1 GB a unos 150 MB), lo que abarata su almacenamiento y acelera enormemente su despliegue cuando necesitas escalar la aplicación. Además, minimizas la superficie de ataque: si un atacante logra vulnerar el contenedor, no tendrá a su disposición compiladores ni herramientas de construcción que le permitan descargar, compilar o ejecutar código malicioso dentro de tu entorno de producción.

¿Que impacto tiene esta decision en el tama no de la imagen y en la superficie de ataque (CVEs)?

Eliminar MVN y JDK reduce el peso final drasticamente

y en los CVES al haber menos software, se eliminan cientos de vulnerabilidades potenciales. Además, sin compiladores a bordo, un atacante infiltrado se queda sin herramientas para crear o descargar malware.

Shift-Left Testing en CI:

2. **Shift-Left Testing en CI:** Explique qué sucede si un desarrollador introduce un cambio que rompe una prueba unitaria en el Backend. ¿Cómo protege el pipeline de GitHub Actions la estabilidad de la rama `main`?

- **Fallo inmediato del Pipeline:** El comando `mvn clean package` (dentro del Dockerfile o en un job de GitHub Actions) detectará la prueba rota. Esto aborta la compilación al instante y marca el Job como fallido (cruz roja).
- **Bloqueo automático del Merge:** Si tienes configuradas las reglas de protección de rama (Branch Protection Rules) en GitHub, un pipeline fallido desactiva automáticamente el botón de "Merge". El código

Pruebas de Humo (Smoke Testing)

3. **Pruebas de Humo (Smoke Testing):** ¿Cuál es la diferencia fundamental entre una prueba unitaria (*Unit Test*) ejecutada en la fase de build y una prueba de humo (*Smoke Test*) ejecutada tras levantar los contenedores?

- **Pruebas Unitarias (Fase de Build):** Validan **el código fuente**. Se ejecutan en el compilador (ej. `mvn test`) antes de empaquetar la aplicación, probando métodos aislados con bases de datos en memoria (H2) o datos simulados (mocks) para asegurar que la lógica matemática o de negocio sea impecable.
- **Pruebas de Humo (Post-Despliegue):** Validan **la infraestructura**. Se realizan sobre los contenedores ya encendidos (ej. un `curl` al healthcheck del backend) para asegurar que el sistema ensamblado realmente arrancó, que los puertos mapearon bien y que el servidor no se apaga repentinamente al recibir tráfico.

Reflexion:

Estas herramientas deberían utilizarse como un apoyo para agilizar las automatizaciones en ci/cd